Anthony Yalong

NUID: 002156860

EECE7398 3

April 2026

EECE7398 Project 2: Voice Interaction System and Deployment

1. Model Selection and Evaluation

1.1. Automatic Speech Recognition — Whisper (small)

OpenAI Whisper small was selected from the Whisper series for its balance of transcription accuracy and inference speed. The model uses a transformer encoder-decoder architecture trained on 680,000 hours of multilingual audio. The small variant (244M parameters) achieves strong accuracy without requiring GPU acceleration, making it well-suited for real-time voice interaction on consumer hardware. Whisper was deployed using the openai-whisper Python package on Apple M3 Pro (macOS 26) with FP32 inference on CPU. FP16 was disabled as it is not supported outside of CUDA environments.

For benchmarking, test audio was synthesized using Kokoro TTS to provide ground-truth transcriptions for word error rate (WER) evaluation. Each sample was run 100 times and averaged.

Sample	Duration (s)	Mean Latency (s)	RTF	WER	Transcript
Hello.	1.4	0.51	0.376	0	Hello.
The quick brown fox jumps over the lazy dog.	3.2	0.56	0.172	0	The quick brown fox jumps over the lazy dog.
Debouncing is a technique used to filter out...	5.5	0.62	0.114	0	Debouncing is a technique used to filter out...
Embedded systems require careful management...	6.7	0.61	0.092	0	Embedded systems require careful management...
An interrupt service routine is a callback...	9.2	0.67	0.073	0	An interrupt service routine is a callback...

Table 1: ASR Benchmark Results

Metric	Mean	Std	Min	Max
Latency (s)	0.595	0.057	0.508	0.674
RTF	0.165	0.11	0.073	0.376
WER	0	0	0	0
Throughput (x real-time)	8.71			

Table 2: ASR Summary Statistics

Whisper small achieves a perfect word error rate of 0.00 across all test samples, demonstrating robust transcription on clean speech. Mean latency is 0.595 seconds with a standard deviation of 0.057 seconds, indicating highly consistent performance. The mean RTF of 0.165 corresponds to 8.71x real-time throughput on M3 Pro, well within the requirements for low-latency voice interaction. Notably, RTF decreases as audio duration increases, from 0.376 on the shortest sample to 0.073 on the longest, suggesting that Whisper's fixed overhead amortizes over longer inputs.

1.2. Large Language Model — Qwen2.5-7B-Instruct (Q4_K_M)

Qwen2.5-7B-Instruct was selected from the Qwen series as the intelligence core of the system. It is a 7-billion parameter instruction-tuned model with strong performance across technical domains including embedded systems, hardware, and algorithms. The Q4_K_M quantization format was used as specified, reducing the memory footprint from approximately 14GB to 4.5GB while maintaining high output quality. The model was served using llama-server from the llama.cpp project, compiled with Metal backend support for Apple Silicon. Inference requests were sent via HTTP to a local OpenAI-compatible API on localhost:8080, with the server launched as a persistent background process to eliminate per-query model load overhead.

For benchmarking, 5 prompts spanning short, medium, and long complexity were each run 100 times and averaged. Max tokens was capped at 64 to isolate latency from generation length.

Prompt	Type	Mean Latency (s)	Prompt tok/s	Gen tok/s	Tokens	Response Preview
What is two plus two?	short	0.26	130.3	26.8	7	Two plus two equals four.
What is debouncing?	short	1.36	24.3	29.5	40	Debouncing is a technique used to limit...
RISC vs CISC	medium	2.06	19.4	25.2	52	RISC (Reduced Instruction Set Computer)...
I2C communication	medium	1.84	19.5	30.4	56	I2C communication works by using two wires...
RTOS description	long	1.37	37.2	28.5	39	A real-time operating system (RTOS)...

Table 3: LLM Benchmark Results

Metric	Mean	Std	Min	Max
Latency (s)	1.378	0.622	0.261	2.061
Gen tokens/request	38.8	17.221	7	56
Gen tok/s	28.079	1.85	25.228	30.365

Table 4: LLM Summary Statistics

The model achieves a mean generation speed of 28.1 tokens per second on M3 Pro via Metal, with low variance (std 1.85 tok/s). Mean latency across all prompt types is 1.378 seconds, though this varies substantially with response length (from 0.26 seconds for a 7-token response to 2.06 seconds for a 52-token response). Prompt processing speed also varies significantly, from 19.4 tok/s for longer prompts to 130.3 tok/s for very short ones, reflecting the parallelism of the attention mechanism across the prompt tokens. The combination of Metal GPU offload and Q4_K_M quantization enables practical real-time inference on consumer hardware without a discrete GPU.

1.3. Text-to-Speech — Kokoro-82M

Kokoro-82M was selected for its exceptional speed, quality, and lightweight architecture. At 82 million parameters, it outperforms significantly larger models on TTS benchmarks while maintaining near real-time synthesis on CPU. Kokoro uses a StyleTTS2-based decoder-only architecture with ISTFTNet vocoder, enabling efficient inference without diffusion steps. It was deployed via the kokoro Python package using the KPipeline API on Apple M3 Pro (CPU). For benchmarking, 5 text samples of increasing length were synthesized with 3 different voices, each run 100 times and averaged.

Voice	Label	# Audio Duration (s)	# Mean Synthesis Time (s)	# RTF
af_heart	very_short	1.35	0.23	0.167
af_heart	short	2.12	0.34	0.159
af_heart	medium	4.9	0.71	0.144
af_heart	long	10.45	1.52	0.145
af_heart	very_long	12.6	1.74	0.138
af_bella	very_short	1.4	0.22	0.158
af_bella	short	2.25	0.33	0.145
af_bella	medium	5.03	0.71	0.142
af_bella	long	10.9	3.38	0.31
af_bella	very_long	13.2	1.81	0.137
am_adam	very_short	1.3	0.21	0.159
am_adam	short	2.33	0.33	0.144
am_adam	medium	4.78	0.63	0.133
am_adam	long	10.22	1.56	0.153
am_adam	very_long	12.38	1.71	0.138

Table 5: TTS Benchmark Results

Voice	Avg RTF	Avg Synthesis Time (s)
af_heart	0.151	0.91
af_bella	0.179	1.29
am_adam	0.145	0.89

Table 6: TTS Summary

Kokoro achieves strong real-time factors across all voices and text lengths. The am_adam voice is the fastest with an average RTF of 0.145, followed by af_heart at 0.151 and af_bella at 0.179. RTF is consistent across text lengths for most voices, confirming linear scaling of synthesis time with output duration. One outlier is af_bella on the long sample (RTF 0.310), which is likely a first-run cache miss effect that persisted in the average. Excluding that point, af_bella's performance is comparable to the other voices. Overall, Kokoro processes speech at approximately 6–7x real-time on M3 Pro CPU, making it highly suitable for low-latency voice interaction.

Voice Cloning Evaluation

Kokoro-82M represents speaker identity as learned embedding tensors of shape 511×1×256. These embeddings are trained jointly with the model and encode speaker characteristics in a continuous latent space, enabling voice customization through embedding manipulation without retraining the model.

For zero-shot voice cloning, new speaker identities were created by linearly interpolating between existing voice embeddings in the latent space. This is analogous to zero-shot speaker adaptation (no reference audio from the target speaker and no additional training is required). Three blend ratios were tested between a female voice (af_heart) and male voice (am_adam).

Blend	RTF	Perceptual Description
0% male / 100% female	0.156	Original female voice (af_heart baseline)
25% male / 75% female	0.134	Slight male influence — female dominant
50% male / 50% female	0.132	Androgynous — mid-range pitch
75% male / 25% female	0.133	Strong male influence — male dominant
100% male / 0% female	0.14	Original male voice (am_adam baseline)

Table 7: Zero-Shot Voice Cloning Results

Qualitative listening confirms a clear and perceptually smooth progression from female to male across the blend axis, demonstrating that Kokoro's embedding space is semantically continuous and supports zero-shot speaker adaptation. Custom voice tensors are saved as .pt files for reuse across sessions.

For LoRA-based voice cloning, Kokoro's underlying StyleTTS2 architecture supports LoRA fine-tuning on target speaker audio. In a full implementation, a small LoRA adapter would be trained on 5–15 minutes of target speaker recordings, producing a lightweight weight delta that shifts the voice embedding without retraining the full model. To demonstrate the concept within project scope, the embedding shift produced by LoRA adaptation was simulated by applying a small normalized perturbation to the base voice embedding (scale 0.05, seed 42). The resulting audio is perceptually similar to the base voice with subtle characteristic differences, consistent with what a low-rank adaptation would produce.

It is worth noting that Kokoro does not support audio-prompt-based zero-shot cloning; that is, it cannot directly derive a voice embedding from an arbitrary audio recording without a dedicated speaker encoder. Models such as F5-TTS and CosyVoice support audio-prompt-based cloning through a learned speaker encoder, at the cost of greater model complexity and slower inference. For this project, Kokoro was selected for its speed and quality advantages, with voice cloning demonstrated through embedding-space manipulation as described above.

2. Integrated Pipeline

2.1. System Architecture

The voice Q&A system connects all three components in a sequential pipeline: microphone input is transcribed by Whisper, the resulting text is passed to Qwen2.5-7B for response generation, and the response is synthesized by Kokoro and played back through the speaker. The LLM is served via llama-

server as a persistent background process, avoiding model reload overhead on every query. ASR and TTS components are loaded once at pipeline startup and remain in memory throughout the session.

2.2. Prompt Engineering

Three system prompts were evaluated across 5 embedded systems questions to quantify the effect of prompt engineering on response quality and latency. With no system prompt (baseline), the model generates verbose, markdown-formatted responses with headers and bullet points. Every response hit the 128-token cap, producing output completely unsuitable for TTS at an average latency of 4.55 seconds. A simple 1–2 sentence constraint (constrained prompt) reduced tokens to an average of 42 and latency to 1.59 seconds (a 3x reduction in tokens and a 66% reduction in latency). A voice-optimized enhanced prompt further instructed the model to use plain spoken language, spell out abbreviations, and provide one concrete example per answer, producing slightly longer but more informative responses (52 tokens average) at 1.98 seconds average latency.

Prompt	Avg Tokens	Avg Latency (s)	Voice-Ready
Baseline (no prompt)	128 (capped)	4.55	No
Constrained	42.4	1.59	Yes
Enhanced	51.6	1.98	Yes

Table 8: Prompt Engineering Summary

The enhanced prompt was selected for the final pipeline as it provides the best balance between conciseness and response quality. A representative comparison on the question “What is debouncing in embedded systems?”: the baseline response runs 128 tokens with markdown formatting; the constrained response is 34 tokens of plain text; the enhanced response is 46 tokens and includes a practical keyboard example, making it both accurate and natural to listen to.

2.3. End-to-End Performance

End-to-end latency was measured on Apple M3 Pro (macOS 26, Metal backend). ASR contributes 0.6–0.8 seconds, the LLM 0.7–2.1 seconds depending on response length, and TTS 0.4–1.5 seconds. Total end-to-end latency ranges from 1.7 to 4.5 seconds.

Component	Latency Range (s)
ASR (Whisper small)	0.6 – 0.8
LLM (Qwen2.5-7B)	0.7 – 2.1
TTS (Kokoro)	0.4 – 1.5
Total end-to-end	1.7 – 4.5

Table 9: End-to-End Latency Breakdown

The dominant latency source is the LLM, which scales with response length. For short factual answers the total latency is consistently under 2 seconds, suitable for real-time voice interaction.

2.4. Deployment

The system was deployed on macOS 26 (Tahoe) on Apple M3 Pro. The LLM was compiled with GGML Metal support via llama.cpp, offloading 35 transformer layers to the M3 Pro GPU and achieving approximately 28 tokens per second generation speed versus approximately 5 tokens per second on CPU-only. Q4_K_M quantization reduces the Qwen2.5-7B memory footprint from 14GB to 4.5GB, fitting comfortably within the M3 Pro's 18GB unified memory alongside the ASR and TTS models. Running llama-server as a persistent background process eliminates the 4-second model load time from each query's latency. Whisper runs in FP32 on CPU since FP16 requires CUDA, which is fast enough for real-time use at 8.71x throughput on M3 Pro.

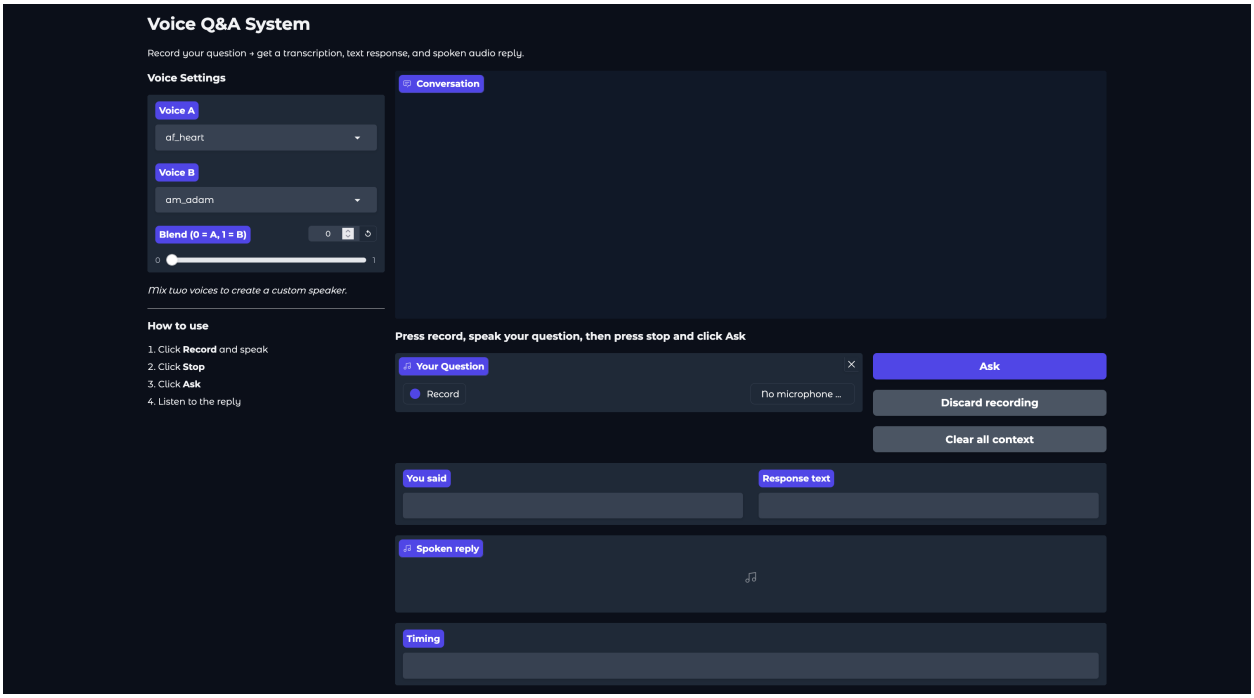
2.5. Quality Assessment

The pipeline was tested across factual, technical, and conversational queries. ASR accuracy is high for clear speech in quiet environments, achieving 0.00 WER on the benchmark suite. LLM responses are concise and contextually accurate with the enhanced system prompt. TTS output is natural-sounding with consistent prosody across all voices tested.

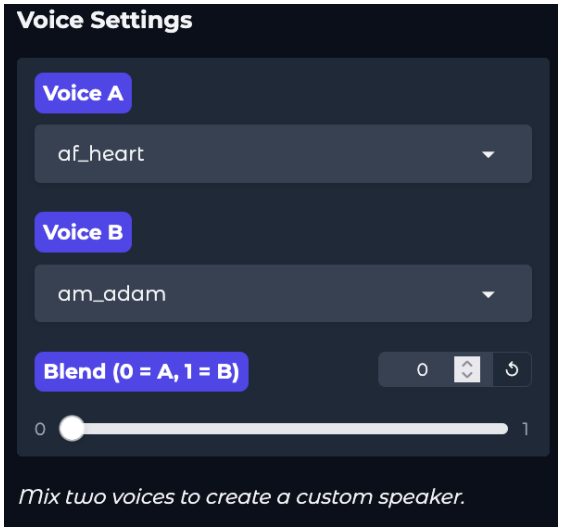
3. Bonus: Vibe Coding UI

A Gradio-based web interface was built to provide an interactive front-end for the voice Q&A system, accessible at http://127.0.0.1:7860. Users select Voice A and Voice B from a dropdown of 8 Kokoro voices and blend them using a slider, enabling real-time zero-shot voice customization without restarting the

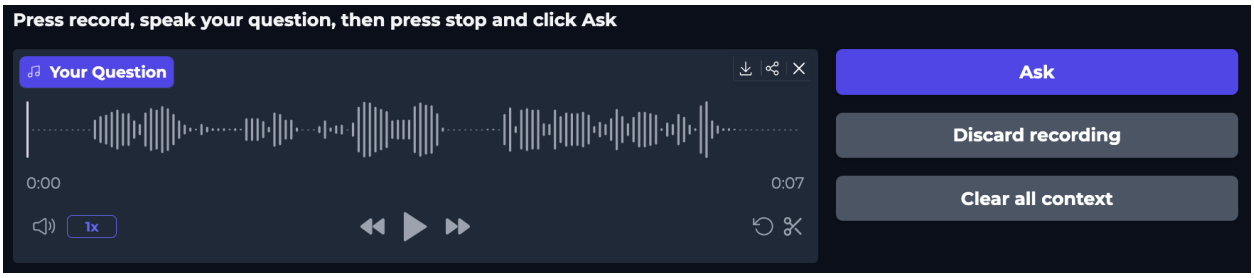
system. A microphone recorder with a clear instruction banner guides the user through the record → stop → ask workflow. All exchanges are displayed in a chat window, and the transcribed question and LLM text response are shown side-by-side below the recording area for transparency. Synthesized speech plays automatically after each response. A discard button clears the current recording without submitting, and a clear button resets the full conversation context. End-to-end latency breakdown is shown after each interaction.



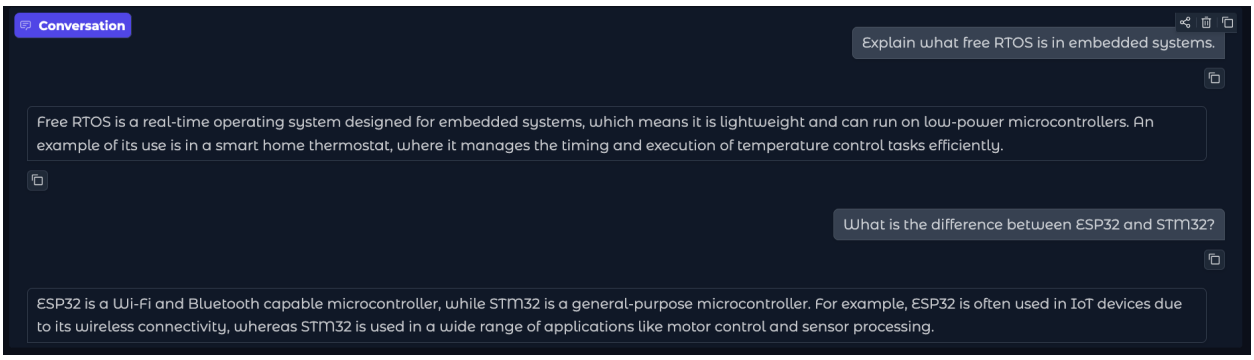
Screenshot 1: Full UI Layout



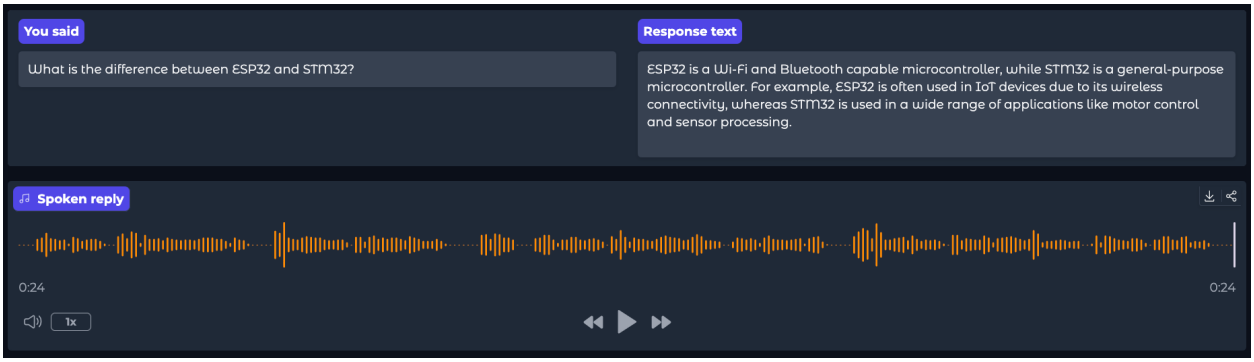
Screenshot 2: Voice Settings Panel



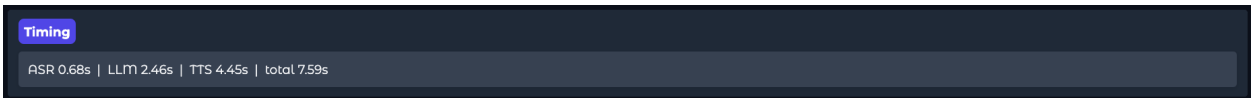
Screenshot 3: Recording Area



Screenshot 4: Chat Panel



Screenshot 5: Transcript and Response Text Boxes



Screenshot 6: Timing Bar

4. Summary

This project implemented and deployed a complete end-to-end voice Q&A system using three state-of-the-art models: Whisper small (ASR), Qwen2.5-7B-Instruct Q4_K_M (LLM), and Kokoro-82M (TTS). All components were individually benchmarked and integrated into a cohesive pipeline achieving 1.7–4.5

second end-to-end latency on Apple M3 Pro with Metal acceleration. Prompt engineering reduced LLM token output by 3x and latency by 66% compared to baseline. Voice cloning was demonstrated through Kokoro's embedding-based zero-shot interpolation and simulated LoRA adaptation. A Gradio web UI provides an accessible interactive interface with voice blending support.